COMSATS University Islamabad,

Sahiwal Campus.

LifeEase - A Smart HealthCare App

By

Muhammad Bilal Anwar	**CIIT/FA22-BCS-064/SWL**
Muhammad Shoaib Arshad	**CIIT/FA22-BCS-080/SWL**

Supervisor

Ms. Raheela Shazadi

Co-Supervisor

Dr. Fakhar Mustafa

Bachelor of Science in Computer Science (2022-2026)

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Bilal Anwar

_____

Muhammad Shoaib Arshad

_____

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) **"LifeEase-A Smart HealthCare App"** was developed by **Muhammad Bilal Anwar(CIIT/FA22-BCS-064/SWL)** and **Muhammad Shoaib Arshad (CIIT/FA22-BCS-080/SWL)** under the supervision of "**Ms. Raheela Shahzadi**" and co supervision of **"Dr. Fakhar Mustafa"** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor
Ms. Raheela Shahzadi
Lecturer
Department of computer science
COMSATS University Islamabad, Sahiwal Campus

External Examiner
Dr. Muhammad Javed Iqbal
Assistant Professor
Department of Computer Science
University of Engineering & Technology, Taxila

Head of Department
Dr. Tariq Ali
Assistant Professor
Department of computer science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In today's fast-paced world, individuals and families increasingly need reliable, always-available support to manage health, track wellness, and respond to medical needs on time. Traditional healthcare is often reactive and fragmented: people rely on memory for medications and appointments, keep health records scattered across apps or papers, and only seek help once symptoms worsen. This leads to missed doses, delayed care, poor follow-up, lack of continuity, and limited visibility for caregivers-especially for seniors, chronic patients, and busy users.

To address these challenges, **LifeEase** is developed as a smart healthcare mobile application designed to make everyday health management simpler, more organized, and more actionable. **LifeEase** works as a personal health companion that helps users keep essential health information in one place, supports healthy routines, and improves preparedness by enabling timely reminders and structured tracking. Instead of relying on manual recall or disconnected tools, **LifeEase** provides a centralized, user-friendly experience focused on improving consistency and convenience in personal healthcare.

LifeEase is built as a modern mobile solution (primarily using Dart/Flutter) enabling a smooth and responsive cross-platform experience. The application is designed to be practical for daily use - allowing users to monitor key health-related activities, maintain records for later reference, and support better decision-making through organized data. With emphasis on usability, **LifeEase** aims to reduce the burden on users by streamlining routine tasks and helping them stay on top of their health goals.

Overall, **LifeEase** offers a proactive approach to personal healthcare by bringing monitoring, organization, and supportive automation into a single smart app-helping users improve adherence, awareness, and confidence in managing their health over time.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **"Ms. Raheela Shahzadi"** and co-supervisor **"Dr. Fakhar Mustafa".** Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Bilal Anwar

Muhammad Shoaib Arshad

Abbreviations

API	Application Programming Interface
SRS	Software Require Specification
SDD	Software Design Description
GUI	Graphical User Interface
IDEs	Integrated development environments

Table of Contents

List of Figures

List of Tables

Chapter 1
Introduction

1 Introduction

LifeEase is a smart healthcare mobile application developed to simplify and improve the way individuals manage their everyday health needs. With increasing lifestyle pressures and growing health concerns, many people struggle with maintaining consistent health routines, keeping personal medical information organized, and getting timely support for decisions such as medication adherence, symptom tracking, and follow-ups. In most cases, these activities are handled manually through memory, paper notes, or scattered digital tools often leading to missed reminders, poor record management, and delayed responses to health issues.

This application is designed to address these challenges by providing a centralized and user-friendly platform where users can manage essential healthcare-related activities in one place. LifeEase helps users stay organized by supporting structured health tracking, maintaining relevant personal health information for future reference, and enabling timely alerts or reminders that assist users in following routines more effectively. By reducing dependence on manual planning and disconnected systems, the app improves consistency and supports a more proactive approach to personal healthcare.

Currently, many users lack a single, dedicated solution that combines routine health management, information organization, and supportive guidance in an accessible way. LifeEase aims to bridge this gap by offering an intuitive experience that can be used by individuals of different ages and technical backgrounds. The app is designed to be simple to navigate while still providing meaningful functionality that supports daily healthcare monitoring and planning.

In essence, LifeEase focuses on enhancing personal healthcare management through convenience, organization, and proactive support. It improves the user's ability to track, maintain, and act upon health-related information, ultimately helping users build better habits and make informed health decisions over time.

1.1 Brief Overview

LifeEase: A Smart Healthcare App project aims to develop a centralized mobile application that helps users manage and monitor their day-to-day healthcare activities in an organized and efficient way. The system is designed to support proactive health management by reducing reliance on

manual record keeping, improving routine adherence (e.g., reminders and tracking), and providing users with quick access to important health-related information when needed.

LifeEase is being developed using modern cross-platform mobile technology (Flutter/Dart) to ensure smooth user experience, high performance, and compatibility across devices. The app is structured to be scalable and maintainable so that additional healthcare features can be integrated in future versions as user needs evolve.

The application is designed to provide a structured workflow for personal health management by allowing users to record and track health-related details such as basic medical information, daily routines, and other relevant activities in one place. Through an intuitive interface, users can monitor their progress, maintain history for future reference, and stay informed through timely reminders and organized health logs.

A key aspect of LifeEase is its focus on usability and accessibility, ensuring that users of different ages and technical backgrounds can easily navigate and benefit from the system. By centralizing essential health management tasks into a single platform, LifeEase improves convenience, supports better consistency in healthy habits, and enhances overall awareness of personal wellbeing.

Overall, LifeEase provides a practical solution for modernizing personal healthcare organization by improving tracking, reducing missed routines, and helping users manage their health more effectively through a smart, mobile-first approach.

1.2 Relevance to Course Modules

The development of LifeEase: A Smart Healthcare App directly applies and integrates key concepts, tools, and practical skills learned throughout the Computer Science degree program, particularly from software engineering, mobile application development, database systems, and human–computer interaction. The project demonstrates how academic knowledge can be transformed into a real-world solution that addresses common challenges in personal healthcare management such as organization, timely tracking, and accessibility of information.

i. Technical Skills

The development of LifeEase: A Smart Healthcare App applies core technical skills gained during academic coursework, particularly in mobile application development and modern software tools.

The app is primarily developed using Flutter (Dart) to build a responsive, cross-platform solution with consistent user experience. Key implementations include UI development, state management, and integration of app features such as health logging and reminder-based workflows. In addition, concepts such as data handling/storage, input validation, and smooth navigation between modules are utilized to ensure the application remains scalable and user-friendly.

ii. Software Engineering Principles

LifeEase is developed by following fundamental software engineering principles, including structured requirement gathering, modular system design, iterative development, and quality assurance practices. Software engineering knowledge supports the project through the complete development lifecycle requirement analysis, architectural design, implementation, testing, and maintenance ensuring the app is reliable, maintainable, and easy to extend. These principles help in organizing the app into manageable components, improving code reusability, minimizing defects through testing, and enabling continuous improvement through future updates.

iii. Database Design

Database management concepts are essential in LifeEase for organizing and preserving user health-related information in a structured and reliable manner. The app requires an effective data design to manage entities such as user profiles, health records (e.g., symptoms/logs), medication or routine schedules, reminders, and historical tracking data. Concepts like normalization are applied to reduce redundancy and maintain consistency, while appropriate indexing and optimized queries support faster retrieval of records (e.g., viewing past logs or upcoming reminders). Maintaining data integrity is critical so that health information remains accurate, consistent, and usable for later reference.

iv. System Architecture

A strong understanding of software architecture and design patterns is important for building LifeEase as a modular, maintainable, and scalable healthcare application. The system is designed by separating key concerns such as UI layer (screens/components), business logic layer (feature rules and validation), and data layer (storage and retrieval) to ensure clean structure and easier maintenance. This modular approach allows different app features such as tracking, reminders, and record management to function as independent components while still working together seamlessly. Clear separation of responsibilities improves code readability, supports future

expansion, and ensures smooth integration between the app interface and underlying data management mechanisms.

v. UI/UX Design

A user-friendly interface is essential for ensuring that LifeEase can be effectively used by people of different ages and technical backgrounds. Concepts from Human–Computer Interaction (HCI) and UI/UX design courses are applied to create an intuitive and accessible experience, including clear navigation, simple input forms, readable layouts, and well-structured dashboards for viewing health information briefly. The design emphasizes minimizing user effort in routine tasks such as adding health logs, setting reminders, viewing schedules, and reviewing past records, ensuring the app remains easy, efficient, and comfortable to use daily.

vi. Project Management:

Project management concepts play an important role in organizing and delivering the LifeEase application in a systematic way. The project follows agile practices, supporting continuous improvement through iterative development and regular feature refinement. This includes requirements prioritization, breaking development into manageable tasks, setting milestones, tracking progress, and validating features through testing and user feedback. Agile planning helps ensure timely delivery of core modules while allowing flexibility to adjust and enhance the app based on evolving requirements and usability considerations.

vii. Networking

LifeEase applies networking concepts to enable reliable and secure communication between the mobile application and any connected services (such as cloud storage, authentication services, or remote databases). Knowledge of HTTP/HTTPS protocols and RESTful APIs is important for handling tasks like sending and retrieving user health records, syncing data across sessions/devices (if supported), and managing real-time updates where required. Networking concepts also support authentication, and authorization flows to ensure only valid users can access their personal health information. Additionally, secure communication practices (e.g., using TLS via HTTPS, proper token/session handling, and safe data transmission) are essential to protect sensitive healthcare-related data and maintain user trust.

1.3 Project Background

The need for LifeEase: A Smart Healthcare App arises from common challenges and inefficiencies people face in managing their personal health in daily life. Personal healthcare management typically involves coordinating multiple activities such as maintaining medical information, following medication or routine schedules, tracking symptoms or health measurements, and remembering appointments or follow-ups. However, the absence of a single, centralized solution often results in disorganized record keeping, missed routines, and limited ability to review health history when needed.

Prior to the motivation for this project, many users relied on manual methods (memory, notebooks, paper prescriptions) or scattered digital tools (separate apps for notes, reminders, and health tracking). This fragmentation often leads to delays in action (e.g., forgetting a dose or missing a follow-up), difficulty in maintaining consistent routines, and poor long-term tracking of health patterns. Users may also struggle to quickly access accurate information during emergencies or medical consultations, especially when records are not organized in one place.

In addition, families and caregivers often face difficulties in ensuring consistent health management for dependents such as elderly family members or individuals with chronic conditions. Without structured tracking and reminder support, health routines become harder to maintain, and monitoring progress over time becomes inconsistent.

In response to these challenges, LifeEase was conceived as a centralized mobile solution that organizes essential health-related information and supports users through structured tracking and timely reminders. The app is designed to modernize personal healthcare organization by simplifying routine management, improving consistency, and enabling users to store and access records for future reference.

By addressing these limitations, the project aims to improve everyday healthcare management, reduce missed routines, enhance awareness of personal wellbeing, and provide a more reliable and organized approach to maintaining health over time.

1.4 Methodology and Software Lifecycle

LifeEase: A Smart Healthcare App is a user-centered project with diverse and evolving functional requirements, including health information management, routine/medication tracking, reminders

and notifications, progress monitoring, and secure handling of sensitive user data. Since user needs can change over time and new healthcare features may be added based on feedback, the project requires a flexible and iterative development approach.

The Agile software development model has been selected for LifeEase because it supports adaptability, collaboration, and incremental delivery. Agile enables continuous feedback from potential users (patients, caregivers, and general wellness users), helping ensure the app remains aligned with real-world healthcare management needs and usability expectations.

Using Agile, LifeEase is developed in modular iterations, which makes it easier to manage complexity and refine features progressively. Core modules such as user profile and basic health record management, logging/tracking features, reminder scheduling, and history/progress views can be implemented step-by-step, tested frequently, and improved in each iteration based on evaluation and user feedback. This approach improves overall quality, reduces risk, and ensures timely delivery of a reliable smart healthcare solution.

Overall, the Agile model supports the development of LifeEase in a controlled yet flexible manner, enabling continuous improvement and timely delivery of a reliable and user-friendly smart healthcare solution.

1.4.1 Rationale behind Selected Agile Model

Agile is a software development methodology that emphasizes adaptability, collaboration, and continuous delivery. It is highly suitable for LifeEase: A Smart Healthcare App because healthcare application requirements often evolve during development as users and stakeholders identify new needs related to usability, data privacy, clinical workflows, and feature completeness (e.g., notifications, telemedicine stability, AI guidance, and record management).

The involvement of multiple stakeholders throughout the development process patients, doctors, and administrators makes continuous feedback essential. Regular feedback helps improve user experience, reduce workflow friction, strengthen security, and ensure that the application supports real-world healthcare interactions such as appointment booking, consultation handling, and record management.

Agile is based on the following principles:

- **Iterative development:**
  LifeEase is developed in multiple iterations, where each cycle introduces or refines

modules such as authentication and role-based routing, doctor discovery, appointment booking/management, medical records, and telemedicine (Agora RTC).

- **Incremental delivery:**
  Functional components are delivered step by step, allowing the team to release usable features early (e.g., login and dashboards first), while continuing to build advanced features such as push notifications (FCM), AI support, and map-based doctor search in later increments.
- **Continuous feedback:**
  Feedback from patients (ease of booking, reminders), doctors (appointment management, consultation flow), and admins (user control and compliance needs) is collected frequently and used to enhance navigation, reliability, and overall workflow efficiency.
- **Adaptive planning:**
  Planning remains flexible to accommodate changing requirements, such as improving privacy and compliance, enhancing data security, refining telemedicine user flow, and adjusting feature priorities based on practical testing and stakeholder expectations.

This Agile-driven approach supports building LifeEase as a scalable, user-centered, and continuously improving healthcare platform.

Chapter 2

Problem Definition

2 Problem Definition

This chapter defines the core problem addressed by LifeEase and outlines the intended outcomes of the proposed solution.

2.1 Problem Statement

LifeEase: A Smart Healthcare App is designed to address the inefficiencies, lack of organization, and poor accessibility commonly found in personal healthcare management. Many individuals manage important health-related activities such as maintaining basic medical information, tracking symptoms or health routines, following medication schedules, and remembering appointments through manual methods or scattered digital tools. This fragmented approach leads to missed reminders, inconsistent tracking, and difficulty retrieving accurate health history when needed.

Users often face challenges in maintaining regular health routines due to busy schedules, lack of structured reminders, and the absence of a centralized place to store and review health information. Keeping records in notebooks, messages, or multiple apps makes it harder to monitor progress over time or share accurate information during medical consultations. As a result, users may experience delays in taking action, reduced adherence to healthy routines, and limited awareness of health patterns.

Another significant issue is the limited support available for organizing health information in a way that is both simple and reliable. Existing solutions may be too complex, not tailored to everyday needs, or focused on only one aspect (e.g., reminders only, or notes only), forcing users to rely on multiple disconnected tools. This reduces efficiency and increases the chance of losing or forgetting critical information.

Currently, many users lack a unified mobile platform that integrates essential personal healthcare functions such as structured tracking, reminder-based routine management, and organized record maintenance within a single, easy-to-use application. This gap highlights the need for a smart and centralized solution that supports proactive healthcare management.

LifeEase aims to resolve these issues by providing a centralized, user-friendly healthcare app that streamlines daily health management, improves routine adherence through reminders, enables organized tracking of health information, and enhances overall awareness by keeping records available for future reference.

2.2 Deliverables and Development Requirements

LifeEase includes a set of deliverables and development requirements necessary to ensure successful implementation, deployment, and long-term maintainability of the application.

- **Role-Based User Interface (UI) Design:** Develop an intuitive, role-based interface for Patients, Doctors, and Admins. The design must support smooth navigation for tasks such as login/signup, doctor discovery, appointment booking, consultation, and admin management features.
- **Authentication & Role Management:** Implement secure authentication using Firebase Authentication, supporting email/password and Google Sign-In, along with forgot-password/reset flow and OTP screen support. Enforce role-based access control (RBAC) and guard routing using go router.
- **Patient Module (Healthcare Consumer Features):** Deliver patient-facing screens and workflows including:
    - Patient dashboard/home
    - Doctor list & search
    - Doctor profile view
    - Appointment booking, confirmation, and appointment history
    - Profile setup (onboarding) and profile view/edit
    - Medication reminder setup (UI + scheduling logic to be integrated)
- **Doctor Module (Healthcare Provider Features):** Deliver doctor-facing screens and workflows including:
    - Doctor dashboard/home
    - Appointment management
    - Patient details view
    - Doctor profile setup and doctor profile view

    - Telemedicine consultation interface

- **Admin Module (System Management Features):** Deliver admin-facing features including:
    - Admin dashboard
    - User management
    - Content management (basic administrative control panels)
- **Appointment Management System:** Implement structured appointment workflows connecting patients and doctors, including appointment creation, scheduling, viewing history, and real-time updates where applicable (Firestore streams).
- **Telemedicine / Real-Time Consultation:** Integrate Agora RTC to enable video consultation features through a dedicated telemedicine call screen and service wrapper.
- **Health Tracking & Medical Records:** Provide mechanisms for managing basic health tracking and medical record screens, with storage and retrieval through Firestore and optional file uploads through Firebase Storage.
- **Notifications:** Implement notification functionality. Current deliverable includes in-app notifications (Firestore-based); further requirement is to extend to push notifications (FCM) for background alerts (appointments/medication reminders).
- **Location & Map Search (Planned/Partial):** Provide doctor discovery via map search screens. Complete the integration using proper Maps functionality (the repo indicates this is currently partial).
- **AI Support Module (Planned/Stub):** Deliver AI-assisted features (e.g., symptom checker / AI guidance). Current codebase contains an AiService skeleton; a key requirement is to complete backend/API integration and ensure safe prompt handling and logging.
- **Data Storage, Security & Privacy:** Ensure protection of sensitive health data through secure authentication, RBAC, secure local storage (Flutter Secure Storage), Firestore security rules, encrypted communication (HTTPS/TLS), and proper validation/sanitization.

- **Cross-Platform Deployment:** Deliver a Flutter-based solution supporting Android, iOS, and Web (and optionally desktop targets as configured). Ensure consistent UI/UX across platforms.

- **Documentation:** Provide complete technical documentation (setup guides such as Firebase setup, environment configuration) and user guides for Patients/Doctors/Admins.

- **Testing & Quality Assurance:** Conduct unit, widget, integration, and manual testing for authentication, routing, appointment workflows, telemedicine, and data storage services.

- **Maintenance & Future Enhancements:** Plan continuous updates for completing remaining tasks (push notifications, offline caching, emergency SOS, payments, AI completion, improved security hardening, etc.).

2.3 Current System

At present, many patients and general users manage healthcare activities using **manual and disconnected methods**, such as phone calls for appointments, paper-based prescriptions, personal notes, and basic alarms for medication reminders. Communication between patients and doctors is often fragmented across different channels (calls, messages, visits), and there is usually **no unified platform** for managing appointments, maintaining medical records, and accessing consultation support in one place.

This unstructured approach creates several practical challenges:

- **Patients** often face difficulties in finding the right doctor, scheduling appointments efficiently, tracking appointment history, and maintaining organized medical records. Medication routines are frequently managed through memory or simple alarms that do not provide structured tracking or history.
- **Doctors** may struggle with consistent appointment organization, efficiently reviewing patient information prior to consultations, and handling follow-up communication in a systematic way.
- **Administrative management** in many small-to-medium healthcare setups is often manual, making user management, content handling, and record consistency difficult to maintain.

Additionally, remote consultation (telemedicine) is not always available in a reliable, integrated manner. Patients may need to use third-party apps for video calls, which are not connected to their appointment or medical record data.

The lack of a centralized digital healthcare application highlights a major gap: users need a single system that integrates role-based access (patient/doctor/admin), appointment management, medical records, health tracking, reminders, and telemedicine. LifeEase aims to address these limitations by providing a unified, secure, and cross-platform solution.

Chapter 3

Requirement Analysis

3 Requirement Analysis

3.1 Use Case Diagrams

In this Use case diagrams, we show the behavior of our system. This enable you to visualize the different types of roles in a system and how those roles interact with the system.

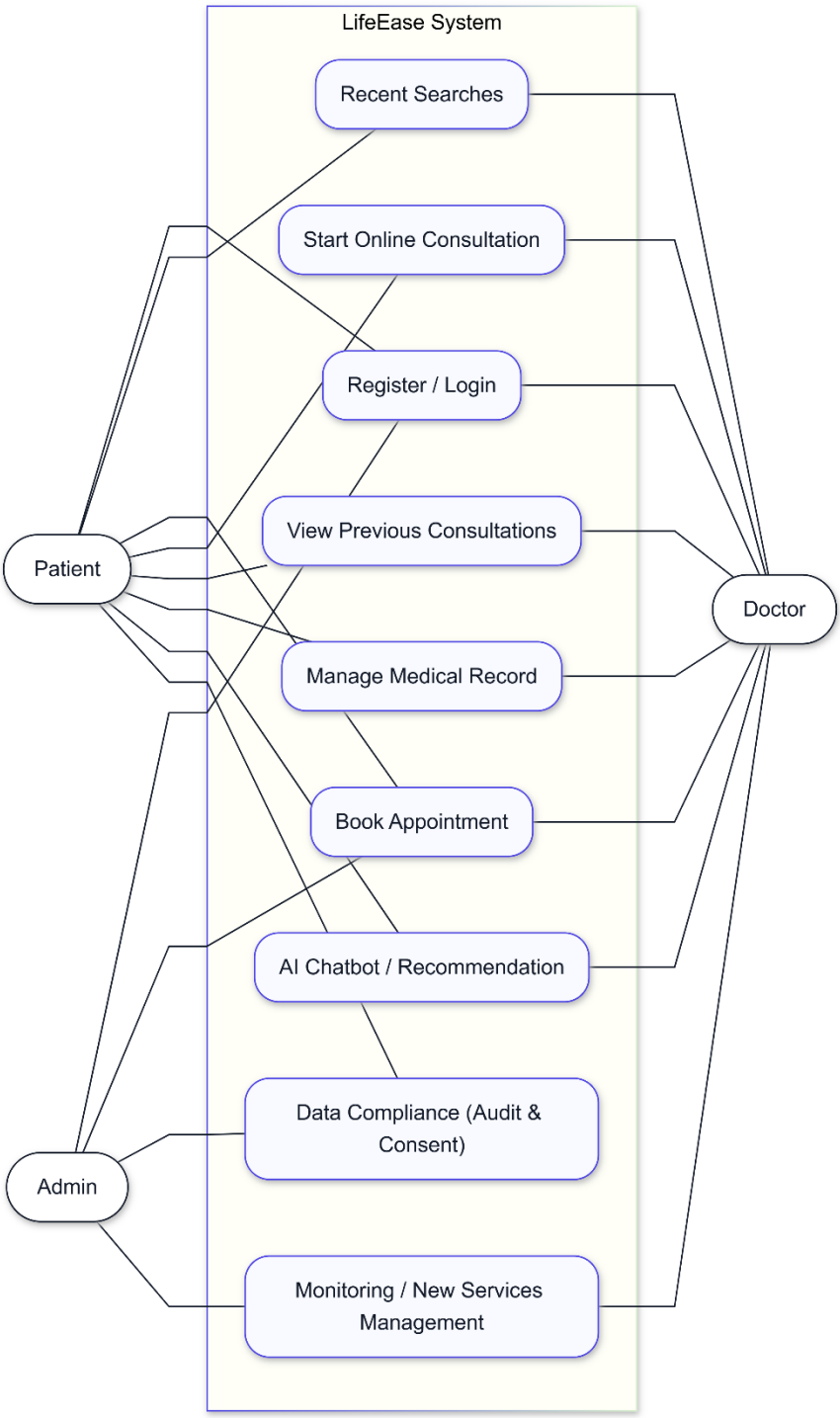

Figure 3-1 LifeEase UseCase Diagram

3.2 Use Case Descriptions

The table below indicate comprehensive use case descriptions

Table 3-1 Use case description for login/logout

Field	**Description**
Use Case ID	UC-01
Use Case Name	Register / Login
Actors	Primary Actors: Patient, Doctor, Admin
Description	This use case defines the authentication process for all system users. It allows users to register (create an account) or log in to access LifeEase features according to their role (Patient/Doctor/Admin).
Trigger	User selects Register or Login on the authentication screen.
Preconditions	User has internet access; for login the user must already be registered; for registration user provides required details.

Table 3-2 Use case description for book Appointment

Field	**Description**
Use Case ID	UC-02
Use Case Name	Book Appointment
Actors	Primary Actor: Patient; Supporting Actors: Doctor (receives request), Admin (override/monitor)
Description	Patient schedules an in-person or online appointment with a doctor by selecting an available time slot. The system stores the appointment and informs relevant actors.
Trigger	Patient selects “Book Appointment” (or proceeds from doctor profile to booking).
Preconditions	Patient is logged in; doctor list/availability is accessible; patient profile is complete (if required).

Postconditions	Appointment created with status (Scheduled/Pending/Confirmed depending on workflow); patient can view it in appointment history; doctor can view it in appointment management; notifications may be generated.

Table 3-3 Use case description for *Manage Medical Record*

Field	Description
Use Case ID	UC-03
Use Case Name	Manage Medical Record
Actors	Primary Actors: Doctor (create/update clinical content), Patient (view); Supporting Actor: Admin (audit/monitor)
Description	Doctors manage patient clinical record entries after consultation, while patients can view their consolidated medical records. The system maintains secure access and audit history.
Trigger	Doctor selects Medical Record for a patient OR patient opens "Medical Records".
Preconditions	Actor is authenticated; patient record exists; proper permissions are granted by role.
Postconditions	Record is created/updated (doctor) OR displayed (patient); changes are logged for auditing.

Table 3-4 Use case description for *View Previous Consultations*

Field	Description
Use Case ID	UC-04
Use Case Name	View Previous Consultations
Actors	Primary Actors: Patient, Doctor
Description	Enables patient and doctor to review past consultation summaries and appointment history for continuity of care.
Trigger	Actor selects "Previous Consultations" or "Appointment History".
Preconditions	Actor is logged in; consultation/appointment history exists in the system.
Postconditions	Historical consultations are displayed; actor can open details of a selected consultation/appointment.

Table 3-5 Use case description for *Start Online Consultation*

Field	Description
Use Case ID	UC-05
Use Case Name	Start Online Consultation
Actors	Primary Actors: Patient, Doctor
Description	Supports real-time online consultation (video call) using telemedicine integration. Both parties join the session through the app at the scheduled time.
Trigger	Patient or Doctor taps “Join Consultation / Start Call”.
Preconditions	Both users are authenticated; a valid appointment/session exists; Agora RTC service and network are available.
Postconditions	Call session starts and ends successfully; session activity can be logged; doctor can proceed to update notes/records if applicable.

Table 3-6 Use case description for *AI Chatbot / Recommendation*

Field	Description
Use Case ID	UC-06
Use Case Name	AI Chatbot / Recommendation
Actors	Primary Actor: Patient; Supporting Actors: Doctor (review), Admin (governance)
Description	Provides AI-based guidance such as symptom-related suggestions or service recommendations. In the current project, the AI service exists as a skeleton/stub and requires full integration.
Trigger	Patient opens AI feature and submits a query.
Preconditions	Patient is authenticated (or allowed guest mode if enabled); AI service/API is configured and reachable (future).
Postconditions	Response is displayed with safety disclaimers; optional logs may be stored based on consent and policy.

Table 3-7 Use case description for *Recent Searches*

Field	Description
Use Case ID	UC-07
Use Case Name	Recent Searches
Actors	Primary Actors: Patient, Doctor
Description	Stores and displays recent search queries for faster doctor discovery and repeated lookups.
Trigger	Actor focuses the search field or opens “Recent Searches”.
Preconditions	Actor is authenticated; local storage is available for saving history (if enabled).
Postconditions	Recent search list is displayed/updated; actor can reuse or clear search terms.

Table 3-8 Use case description for *Data Compliance (Audit & Consent Management)*

Field	Description
Use Case ID	UC-08
Use Case Name	Data Compliance (Audit & Consent Management)
Actors	Primary Actor: Admin; Supporting Actors: Patient (consent view), Doctor (limited access)
Description	Admin reviews audit logs and manages consent/retention policies to support privacy and compliance requirements for sensitive healthcare data.
Trigger	Admin opens compliance/audit dashboard.
Preconditions	Admin is authenticated and authorized (elevated role).
Postconditions	Audit logs reviewed/exported; consent/retention changes applied and recorded.

Table 3-9 Use case description for *Monitoring / New Services Management*

Field	Description
Use Case ID	UC-09
Use Case Name	Monitoring / New Services Management

Actors	Primary Actor: Admin; Supporting Actors: Doctor (advisory), Patient (consumer)
Description	Admin monitors system activity and manages service catalog items (add/edit/disable services) so users can access updated offerings.
Trigger	Admin selects “Monitoring / New Services”.
Preconditions	Admin is authenticated; service catalog/metrics source is available.
Postconditions	Metrics are displayed; service catalog updates are saved and made visible according to publishing rules.

3.3 Functional Requirements

Functional requirements define the core features and operations that the system must provide to enable users to perform their tasks effectively. [1]. LifeEase is composed of multiple modules designed to support **Patients, Doctors, and Admins** in managing healthcare activities such as authentication, appointment scheduling, consultation handling, medical record management, and system administration.

This system will have the following modules:

3.3.1 User Registration

Users must be able to register in the system using valid credentials (email/password) or supported providers such as Google Sign-In. During registration, the user selects a role (Patient/Doctor/Admin) and the system stores profile information for role-based access.Login / Authentication

3.3.2 Login / Authentication

Registered users must be able to securely log in using email/password or Google Sign-In. The system must authenticate the user and enforce role-based access control (RBAC) so that each role can only access permitted screens and features.

3.3.3 Role-Based Dashboard and Navigation

After login, the system must route users to the correct dashboard based on their role:

- Patient Dashboard
- Doctor Dashboard

- Admin Dashboard
  Routing must be protected using guarded navigation so unauthorized roles cannot open restricted modules.

3.3.4 Doctor Search and Discovery

Patients must be able to search and browse doctors using lists and filters. The system should display doctor profiles, specialties, and basic details to support decision making before booking an appointment.

3.3.5 Appointment Booking and Scheduling

Patients must be able to book appointments (in-person or online where supported) by selecting an available doctor and time slot. The system must store appointment details and allow both patients and doctors to view appointment schedules and history.

3.3.6 Appointment Management (Doctor)

Doctors must be able to view appointment requests, accept or reschedule appointments, and update appointment status. The system should reflect updates to the patient in real time where applicable.

3.3.7 Telemedicine / Online Consultation

LifeEase must provide an online consultation feature that enables real-time video consultation between patients and doctors using a telemedicine module (Agora RTC integration). The system should allow joining/leaving sessions securely and maintain session records where required.

3.3.8 Health Tracking and Logs

Patients must be able to add and view health tracking records (such as basic logs, symptoms, or health-related entries). The system must store these entries and display history for monitoring and review.

3.3.9 Medical Records Management

LifeEase must allow patients to view their medical record information and allow doctors (authorized) to create or update clinical entries. The system should support document upload and viewing (where enabled) to maintain consolidated medical documentation.

3.3.10 Notifications and Reminders

The system must notify users about relevant updates (e.g., appointment confirmations/changes). LifeEase must also support reminder setup (e.g., medication reminders) with in-app notification support and future capability for push notifications.

3.3.11 Admin Management Functions

Admins must be able to manage users, review system activity, and maintain app content settings. The system should provide administrative dashboards for monitoring and management workflows.

3.3.12 AI Chatbot / Recommendation (Planned/Stub)

The system may provide AI-assisted guidance (e.g., symptom checker or recommendations). In the current scope, this module is included as planned/stub and should be implemented with safety disclaimers and controlled usage in future versions.

3.4 Nonfunctional Requirements

Non-functional requirements define the quality attributes and constraints of the system, ensuring its performance, reliability, and usability under real-world conditions.

3.4.1 Performance

The system should respond quickly to user actions such as login, doctor search, appointment creation, and record retrieval. The app should efficiently handle multiple concurrent users and remain responsive across supported platforms.

3.4.2 Security

Strong security mechanisms must be implemented to protect sensitive healthcare data and user credentials. The system must ensure:

- secure authentication,
- role-based access control,
- encrypted communication (HTTPS/TLS),
- secure storage where required,
- and enforcement of backend security rules.

3.4.3 Availability

The system should be available whenever users need it, especially for appointment schedules and telemedicine access. Backend services must be reliable, and the system should handle failures gracefully (e.g., retry prompts, safe error handling).

3.4.4 Ease of Use

The app should provide an intuitive, user-friendly interface for patients, doctors, and admins. Navigation must be simple and consistent, with clear dashboards and minimal user effort for common actions such as booking appointments, viewing history, and managing records.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

The structure of the system explains the working of the system and describes the components of LifeEase-A Smart HealthCare App, their relationships, and how they interact with each other, with this architecture [2].

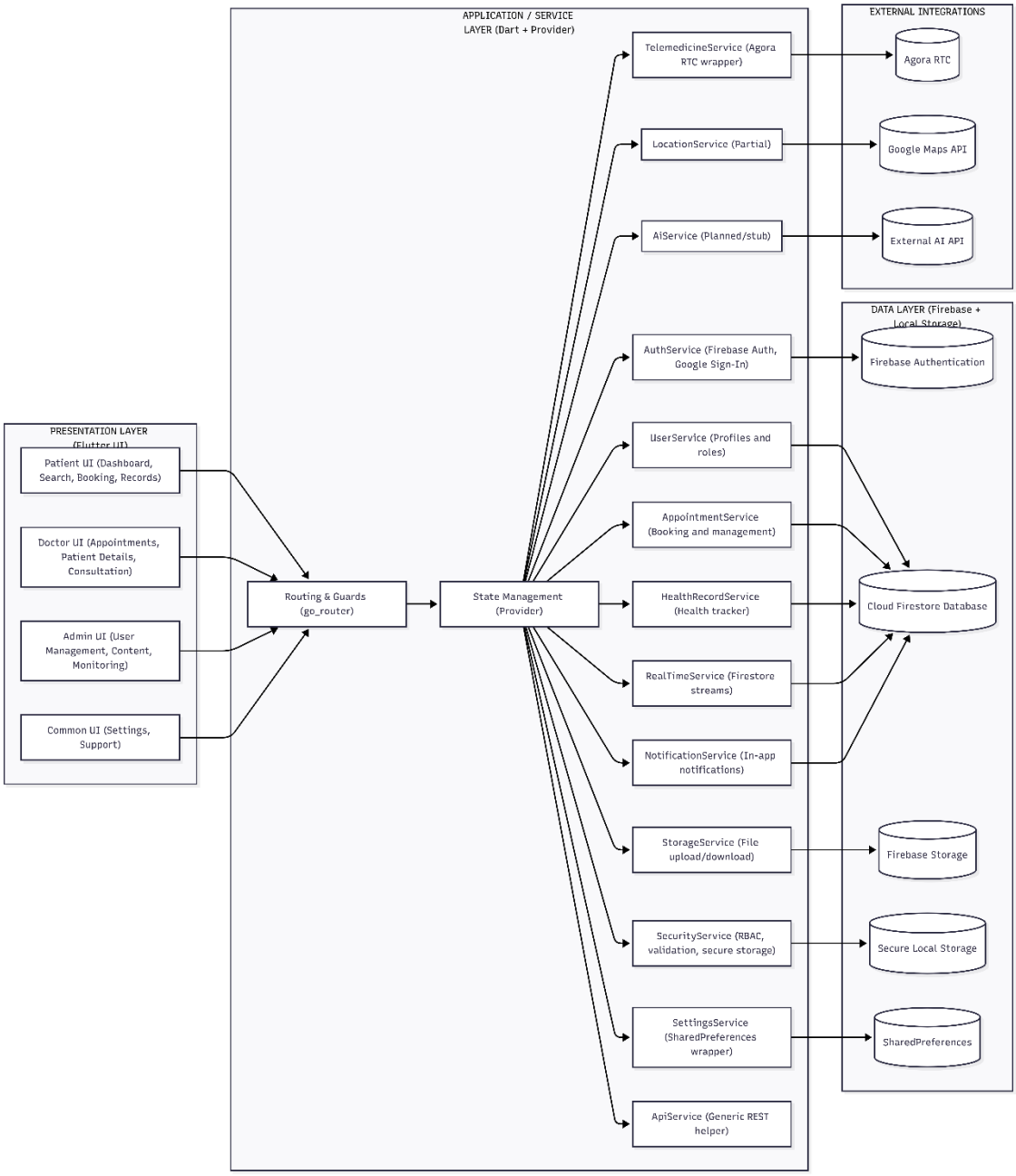

Figure 4-1 System architecture

4.2 Process Flow/Representation

The Process Flow Representation illustrates the step-by-step sequence of actions and interactions between different components in the CUI internship Management System.

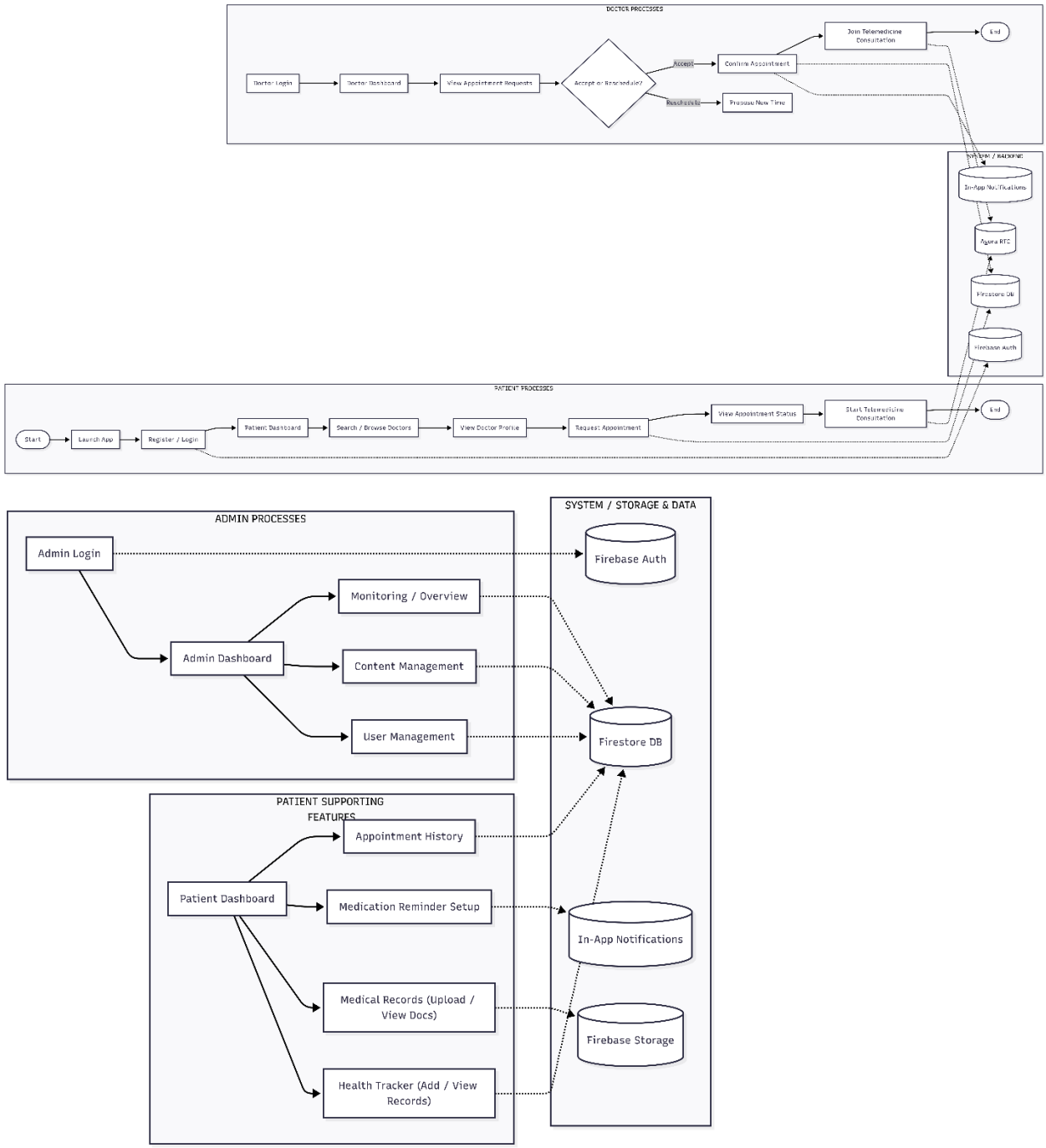

Figure 4-2 Process Flow Diagram

4.3 Sequence Diagram

These figures show how the system will work through sequence diagrams, task after task sequence wise. It gives a complete description of the work of LifeEase-A Smart HealthCare App. [3]

4.3.1 User Authentication Module (Register/Login + Role Routing)

The User Authentication module enables Patients, Doctors, and Admins to securely register or log in using valid credentials and establishes an authenticated session.

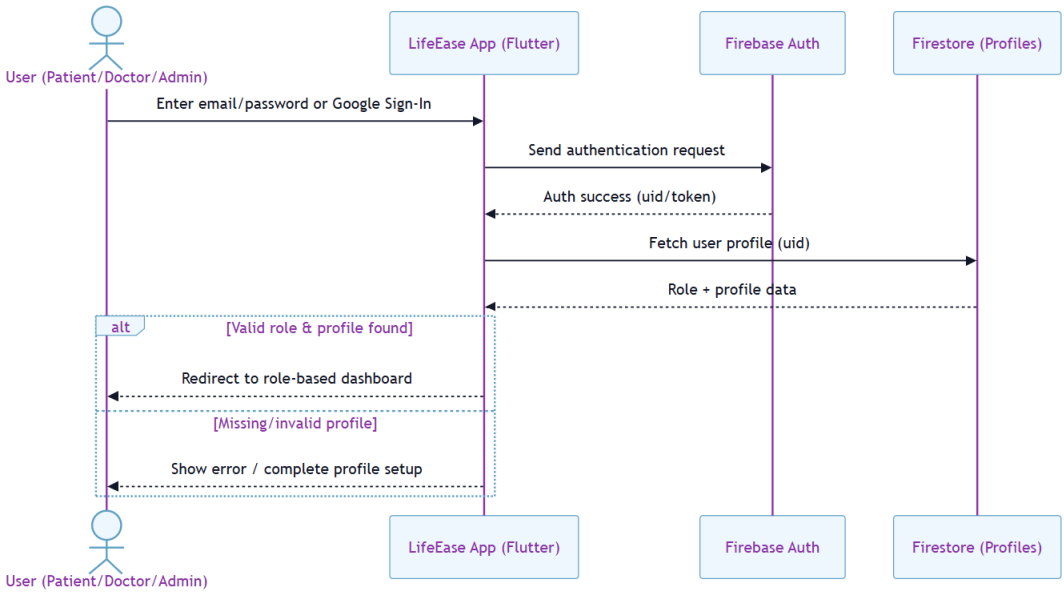

Figure 4-3 Authentication Sequence Diagram

4.3.2 Patient Appointment Booking Module

The Patient Appointment Booking module allows a patient to search or browse doctors, view doctor details, and select an available time slot to schedule an in-person or online appointment.

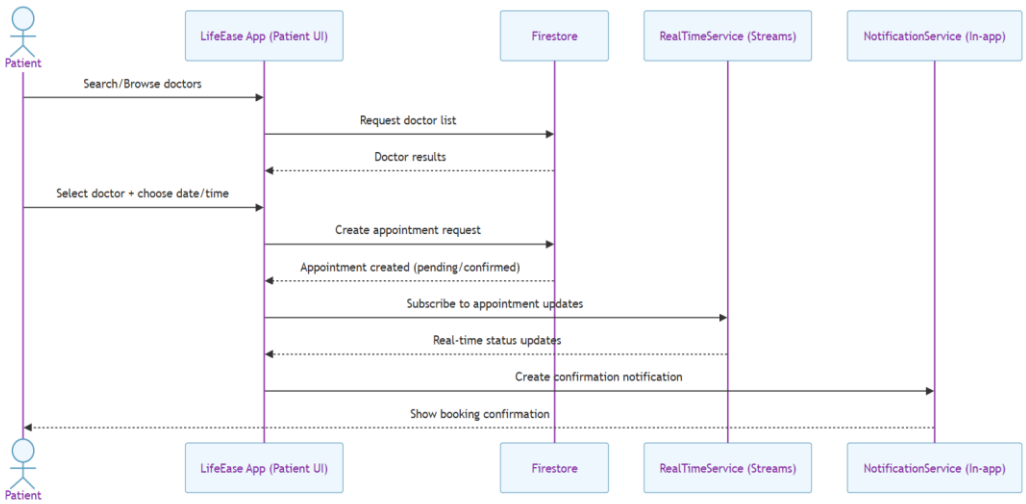

Figure 4-4 Patient Appointment Booking Diagram

4.3.3 Doctor Appointment Management Sequence Module

This module enables doctors to view scheduled and pending appointments, and to accept, reschedule, or update appointment status. Any changes are saved to the system and reflected to patients through updated appointment details and notifications.

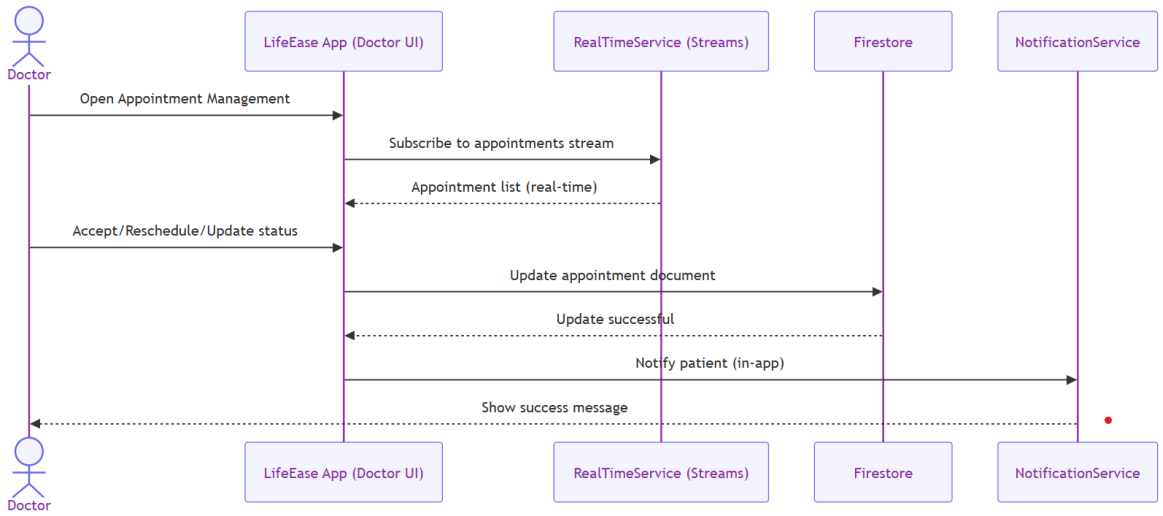

Figure 4-5 Doctor Appointment Management Sequence Diagram

4.3.4 Health Tracker Module (Add / View Health Records)

This module allows patients to enter health-related metrics/logs and maintain a structured history of records over time. The system stores the entries securely and displays trends or previous logs for monitoring and review.

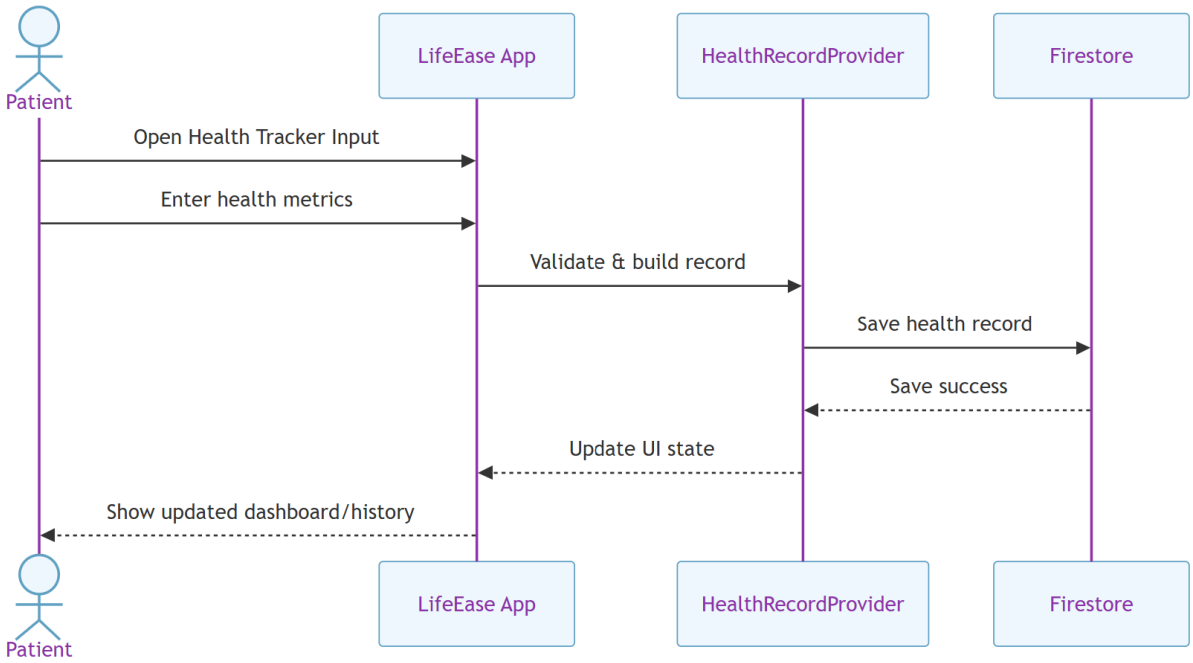

Figure 4-6 Health Tracker Sequence Diagram

4.3.5 Medical Records (Upload/View) Module

This module enables users to upload and store medical documents (e.g., reports, prescriptions) and view them later when needed. The system saves document files in storage and maintains metadata so records remain organized and easily accessible.

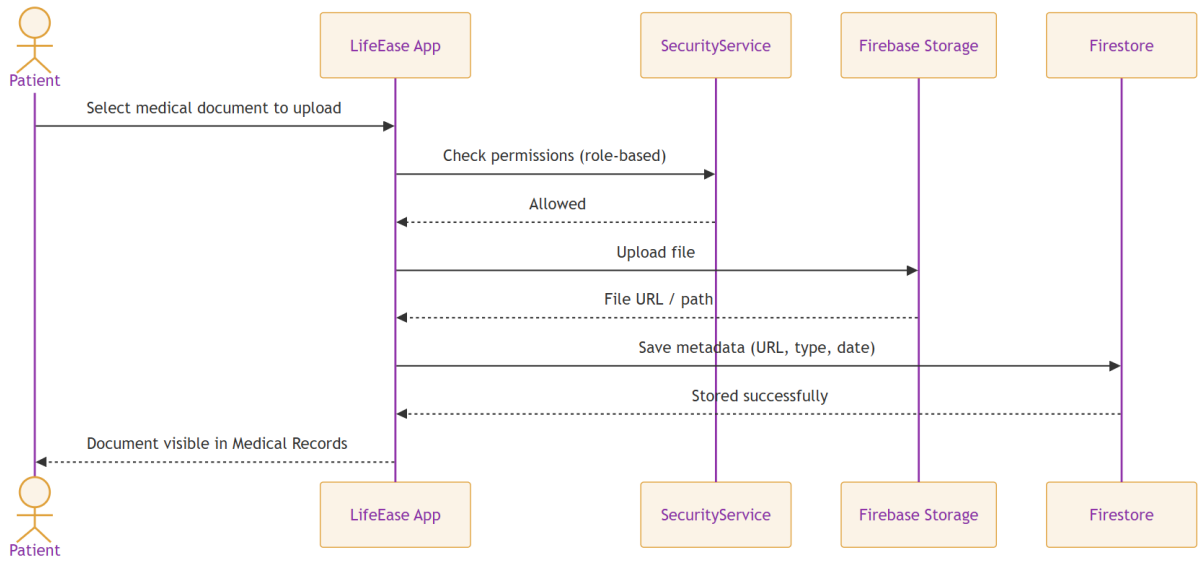

Figure 4-7 Medical Records Sequence Diagram

4.3.6 Telemedicine (Agora RTC) Module

This module provides real-time video consultation between patients and doctors through Agora RTC integration. It manages session initialization, joining/leaving calls, and ensures a smooth consultation workflow within the application.

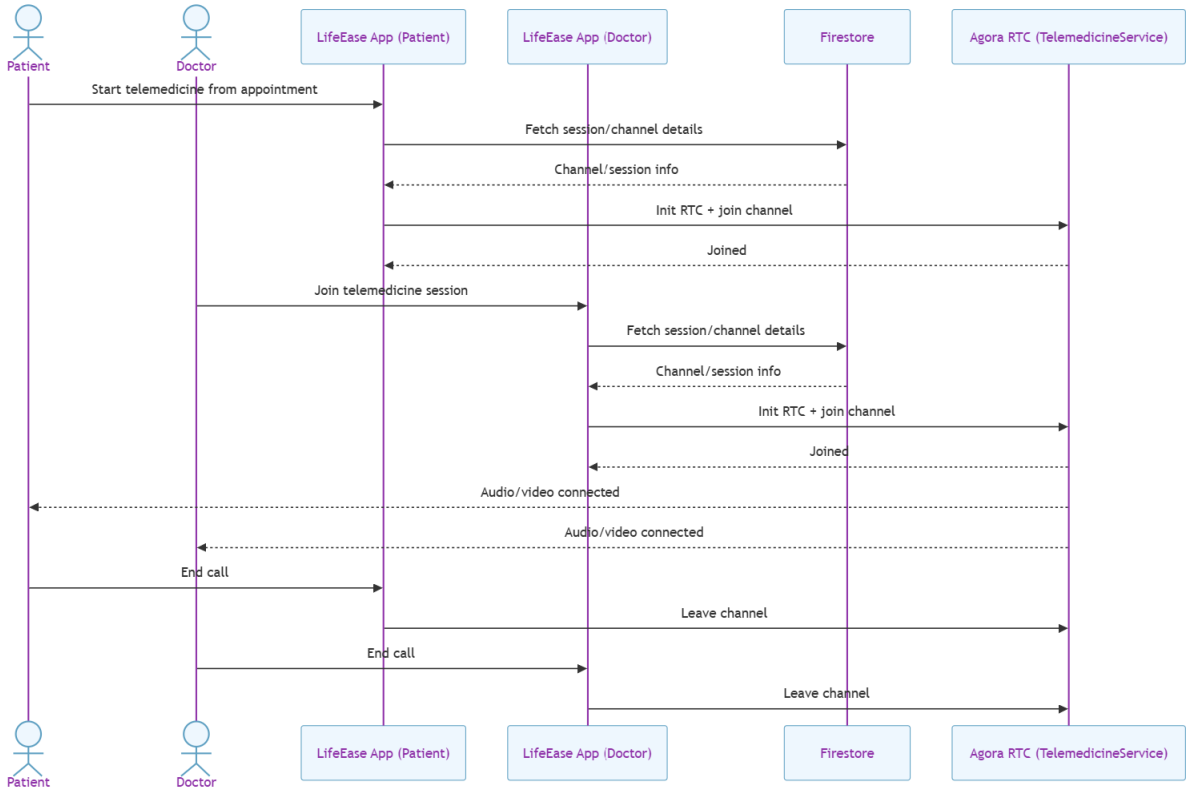

Figure 4-8 Telemedicine (Agora RTC) Sequence Diagram

4.3.7 Admin User Management Module

This module allows administrators to manage system users by viewing user lists, updating profiles/roles, and controlling access when required. It supports administrative monitoring to maintain system integrity and proper role-based usage.

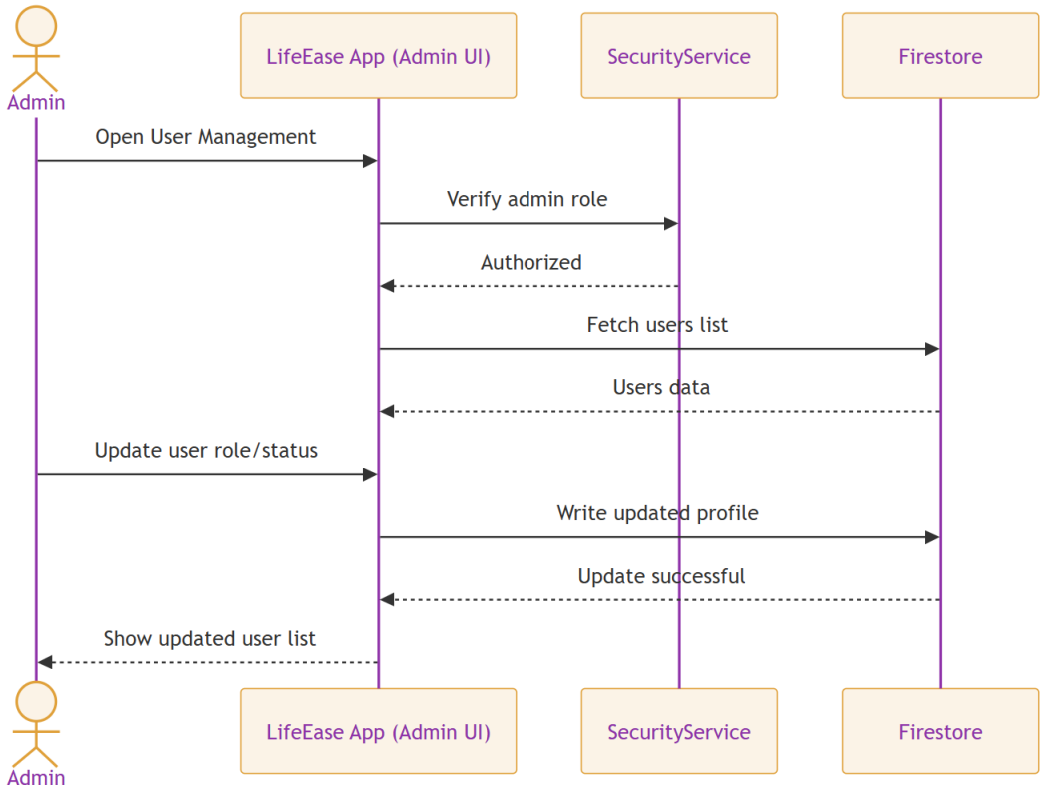

Figure 4-9 Admin User Management Sequence Diagram

4.3.8 AI Support (Planned/Stub) Module

This module is intended to provide AI-based guidance such as basic symptom support or healthcare recommendations for users. In the current version, it is included as a planned/stub component and will require future integration with a secure AI backend and safety controls.

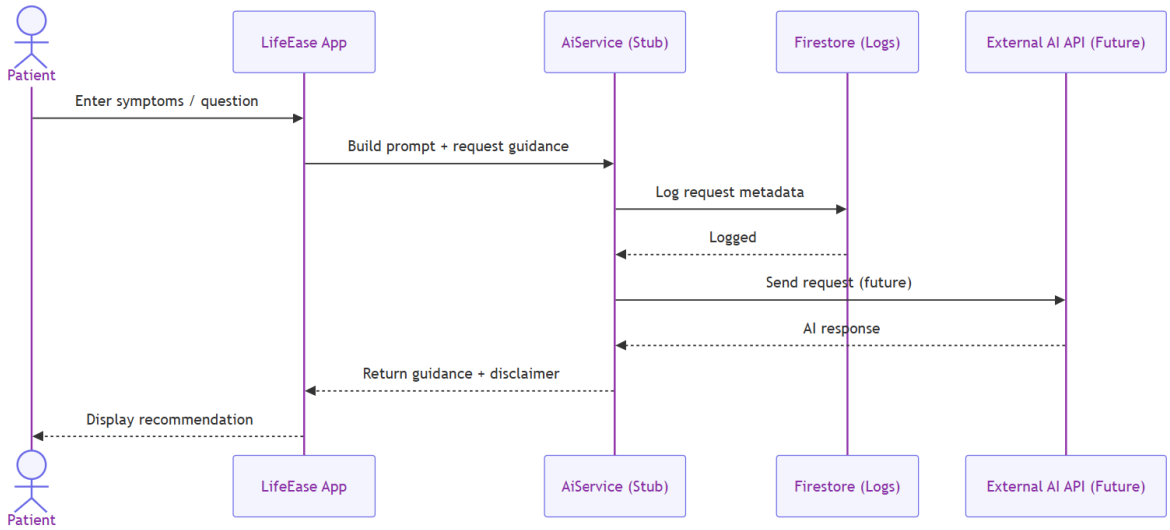

Figure 4-10 AI Support Sequence Diagram

4.4 Class Diagram

This class diagram is a graphical representation of the system's classes, their attributes, and the relationships between them.

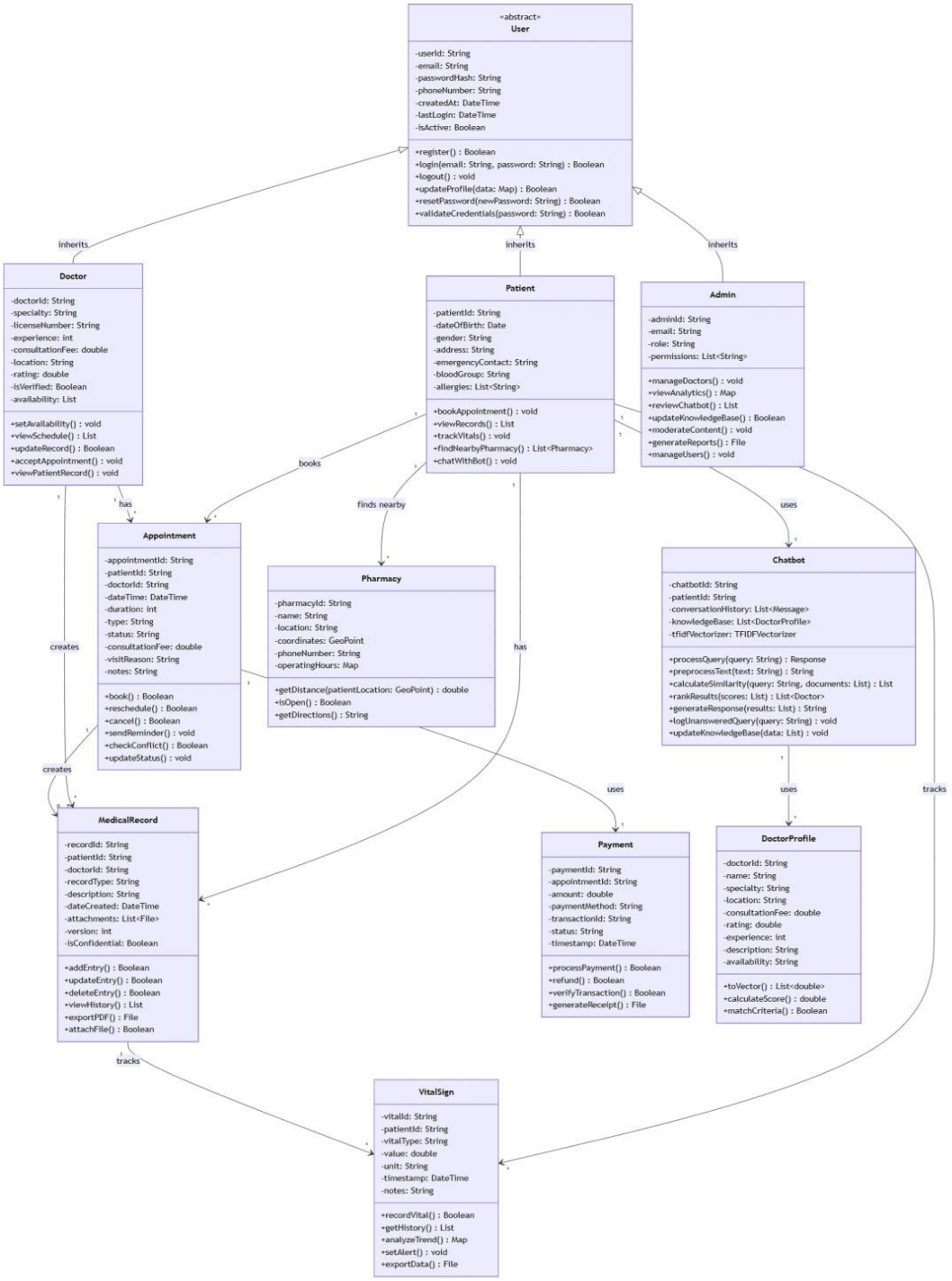

Figure 4-11 Class Diagram

Chapter 5

Implementation

5 Implementation

In the implementation phase of LifeEase, the core modules and services are developed to support smooth interaction between patients, doctors, and administrators. The system focuses on secure authentication, role-based navigation, healthcare data management, appointment workflows, and real-time telemedicine support. The application is implemented using Flutter (Dart) with a modular architecture, using Provider for state management and Firebase services for backend functionality.

5.1 Firebase-Based User Authentication (Role-Based Access)

LifeEase uses Firebase Authentication to provide secure login and authorization. Users can authenticate using Email/Password and Google Sign-In. After login, the system retrieves the user's profile and role (Patient/Doctor/Admin) from Firestore and then applies role-based routing using go_router guards.

This approach ensures:

- secure access to protected screens,
- separation of features by role (Patient vs Doctor vs Admin),
- scalable authentication without manual session handling.

5.2 User Registration Algorithm (Role Selection + Profile Creation)

This algorithm manages the signup process for new users. During registration, the user selects a role (**Patient / Doctor / Admin**) and provides required credentials. The system validates the input, creates the account through Firebase Auth, and stores user profile data in Firestore using the User Service.

Key outcomes:

- Account creation in Firebase Auth
- Role and profile data persisted in Firestore
- Consistent onboarding flow (including profile setup screens)

5.3 Appointment Booking & Management Algorithm

This algorithm governs the appointment workflow between patients and doctors. Patients can browse/search doctors, view doctor profiles, and initiate appointment booking. Appointment

details are stored in Firestore and are accessible through appointment history screens and doctor appointment management screens.
The workflow supports:

- appointment creation and confirmation screens,
- appointment history viewing for patients,
- appointment management interface for doctors,
- real-time updates using Firestore streams (where enabled through the Realtime Service).

5.4 Telemedicine (Agora RTC) Consultation Module

A key component of LifeEase is telemedicine support. The system integrates **Agora RTC** through the TelemedicineService, enabling real-time video consultation between doctors and patients using dedicated call screens.

This module handles:

- initializing the Agora RTC engine,
- joining/leaving consultation sessions,
- supporting remote consultation workflows within the app environment. [4].

5.5 Health Tracking & Medical Records Management

LifeEase includes implementation for patient health-related data handling such as health tracker dashboards/input screens and medical record screens. Data is managed through models and providers (e.g., Health Record Provider) and stored in Firestore. For file-based records (if used), **Firebase Storage** is utilized via Storage Service.

This improves:

- structured storage of user health records,
- easy retrieval and display of health history,
- maintainable data models for future feature expansion.

5.6 User Interface (Multi-Role UI)

The system provides a clean and structured UI for all roles:

- **Patient UI:** dashboards, doctor discovery, appointment booking flow, health tracker, medication reminder UI, medical records, telemedicine screen, and profile management.
- **Doctor UI:** dashboard, appointment management, patient details, profile setup/view, telemedicine consultation.
- **Admin UI:** dashboard, user management, content management.

The UI focuses on usability, clarity, and smooth navigation through a role-based layout using go router, reusable widgets, and consistent theming (light/dark theme support is included).

The Login Screen [Figure 5-1] allows users to securely access the LifeEase application by entering their email/phone number and password. It also provides role-based login options for patients, doctors, and administrators, along with a password recovery feature.

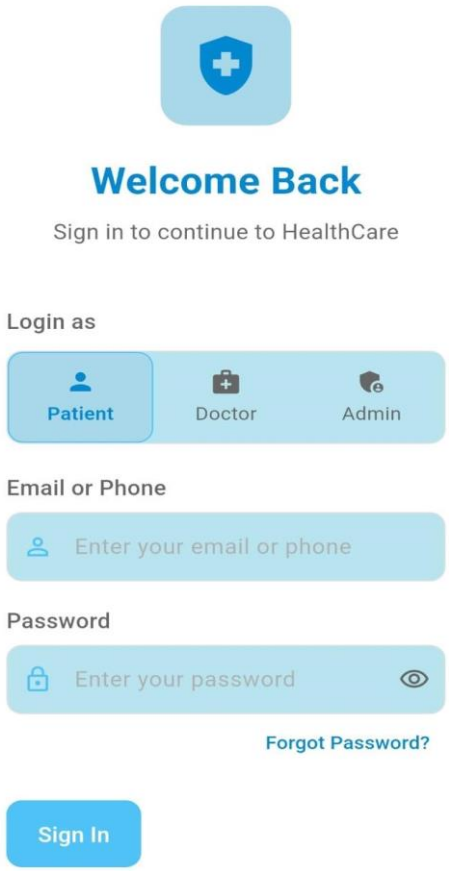

Figure 5-2 LifeEase Log in Screen

The Sign-Up Screen [Figure 5-3] enables new users to create an account by providing personal information, selecting their role, and uploading a profile picture. This ensures a personalized and secure registration process.

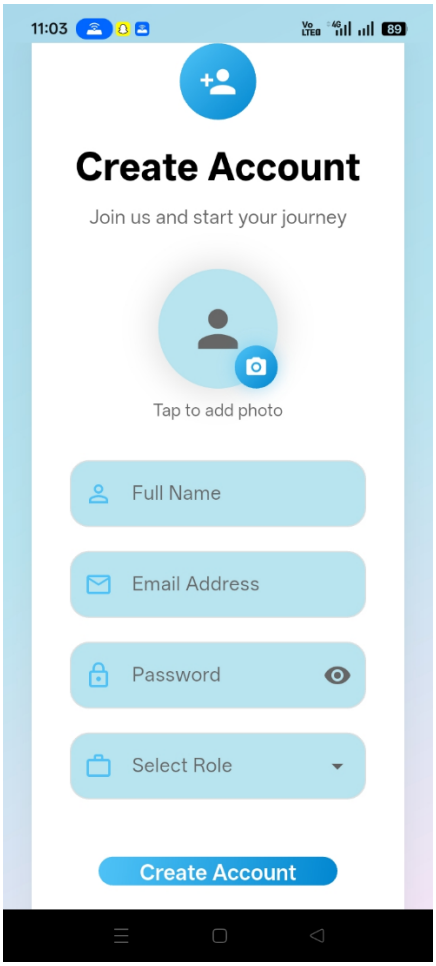

Figure 5-4 LifeEase Sign Up Screen

The Home Screen [Figure 5-3] serves as the user's dashboard, displaying appointment statistics, quick actions, and important healthcare information. It provides easy navigation to the application's core features through an intuitive interface.

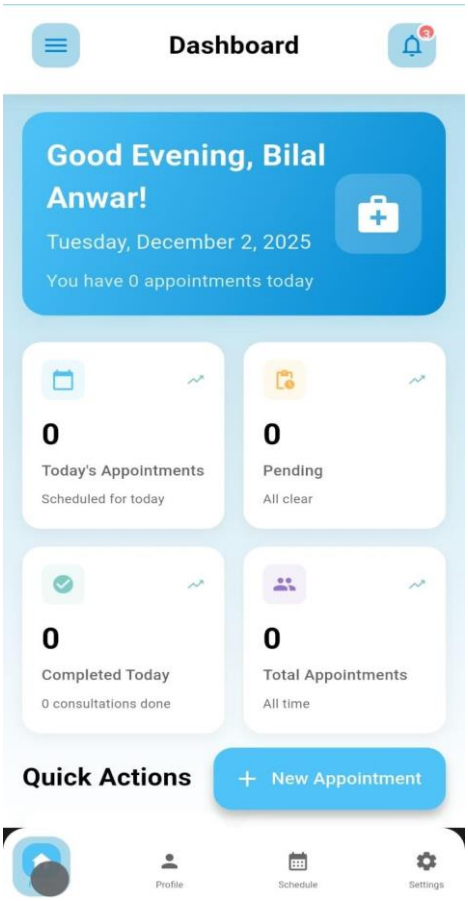

Figure 5-5 LifeEase Home Screen

The Home Screen [Figure 5-4] serves as the user's dashboard, displaying appointment statistics, quick actions, and important healthcare information. It provides easy navigation to the application's core features through an intuitive interface.

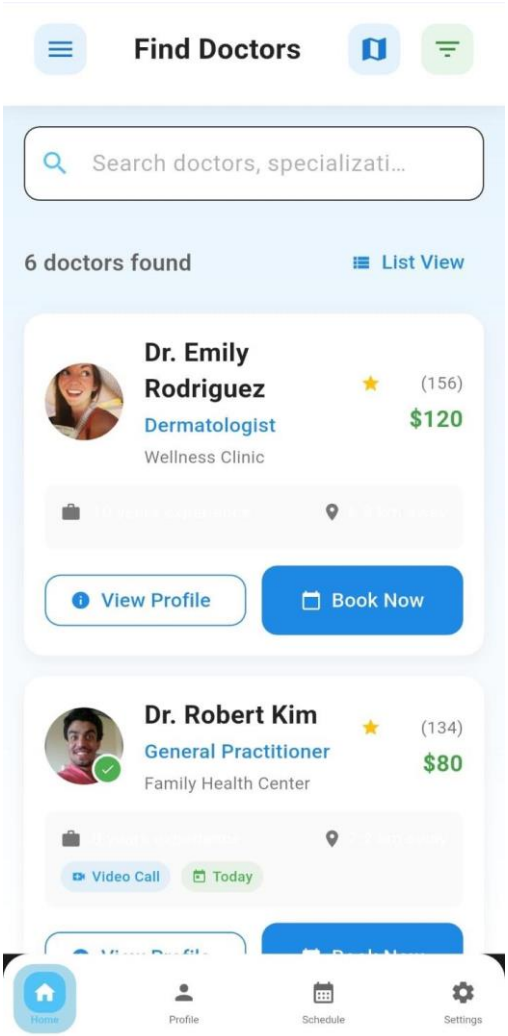

Figure 5-6 LifeEase Find Doctors Screen

The AI Symptom Checker [Figure 5-5] enables users to enter their symptoms and receive an AI-generated preliminary health assessment. It also includes a medical disclaimer reminding users that the results are for informational purposes only and should not replace professional medical advice.

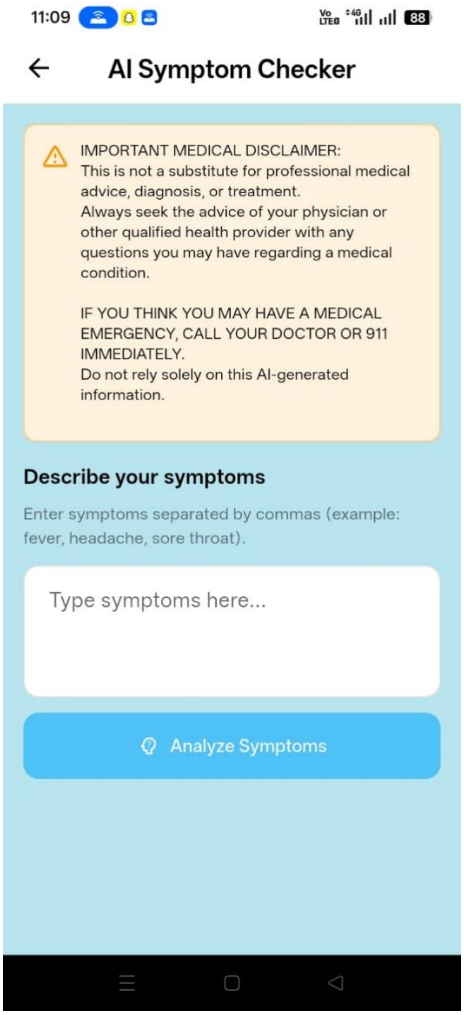

Figure 5-7 LifeEase AI Chatbot Screen

Chapter 6

Testing and Evaluation

6 Testing

6.1 Unit Testing

Unit testing was conducted to evaluate the individual components of **LifeEase** and ensure that each feature performs as expected independently. The purpose was to validate core modules such as authentication, reminders, logging, and data handling in isolation. [5].

Table 6-1 Unit Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Sign Up (Role Selection)	Valid email/password + role (Patient/Doctor/Admin)	Account created and role stored	Pass
UT-002	Login (Email)	Valid credentials	Successful login	Pass
UT-003	Google Sign-In	Valid Google account	Successful login and routing	Pass
UT-004	Forgot Password	Valid registered email	Reset request initiated successfully	Pass
UT-005	Role-Based Routing	Patient/Doctor/Admin roles	Redirect to correct dashboard	Pass
UT-006	Appointment Create (Model/Provider)	Valid appointment fields	Appointment object saved/updated	Pass

6.2 Manual Testing

During the manual testing phase, **LifeEase** was thoroughly tested to verify user flows, usability, navigation, and overall app behavior. Any identified issues were resolved to ensure a smooth and user-friendly experience.

Table 6-2 Manual Testing

Test Cases	Expected Result	Actual Result	Pass/Fail
App Launch	Successful	Successful	Pass
Signup with Role	Successful	Successful	Pass
Login (Email/Google)	Successful	Successful	Pass
Role Dashboard Loads	Accurate	Accurate	Pass
Patient: Doctor List & Search	Successful	Successful	Pass
Patient: Doctor Profile View	Successful	Successful	Pass
Patient: Appointment Booking UI Flow	Successful	Successful	Pass
Patient: Appointment History View	Successful	Successful	Pass
Doctor: Appointment Management Screen	Successful	Successful	Pass
Telemedicine Call Screen Opens	Successful	Successful	Pass
Admin: User Management Screen	Successful	Successful	Pass
Admin: Content Management Screen	Successful	Successful	Pass
Profile View/Edit	Successful	Successful	Pass

6.3 Functional Testing

Functional testing was performed to verify complete app functionalities against the defined requirements and ensure that each feature behaves correctly from an end-user perspective.

Table 6-3 Functional Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
FT-001	User Registration	Valid credentials + selected role	User created successfully	Pass
FT-002	User Login	Valid credentials	User authenticated successfully	Pass
FT-003	Patient Appointment Booking	Select doctor + choose date/time	Appointment request created	Pass
FT-004	Appointment Confirmation Screen	Booking details rendering	Details shown correctly	Pass

FT-005	Appointment History	Previously booked appointments	History displayed correctly	Pass
FT-006	Doctor Appointment Management	View/manage appointment list	Data shown correctly	Pass
FT-007	Medical Records Screen	Retrieve stored records	Records displayed correctly	Pass
FT-008	Telemedicine Consultation	Start/join call	Video call session starts	Pass

6.4 Integration Testing

Integration testing was performed to validate interaction between different modules of LifeEase, ensuring smooth data flow and reliable feature collaboration (e.g., reminders using stored schedules, logs appearing in history views, and authentication controlling access).

Table 6-4 Integration testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
IT-001	UI–Firebase Auth Integration	Sign up/login flows	Auth works and session persists	Pass
IT-002	Firestore User Profile Integration	Store/retrieve role + profile	Profile stored and fetched correctly	Pass
IT-003	Appointment (UI–Firestore) Integration	Booking and fetching appointments	Correct data exchange	Pass
IT-004	Storage Integration	Upload and fetch medical record files	Files stored and accessible	Pass
IT-005	Telemedicine (UI–Agora) Integration	Video call screen with Agora engine	Call initializes and connects	Pass

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

LifeEase: A Smart Healthcare App represents a significant step toward modernizing and simplifying healthcare access and management through a single, role-based digital platform. The project addresses common problems in traditional healthcare interaction, such as fragmented communication between patients and doctors, difficulty in managing appointments, lack of organized medical records, limited remote consultation options, and the absence of a unified system for tracking health-related activities.

LifeEase provides a centralized and structured solution for Patients, Doctors, and Administrators to interact efficiently within one application. Through its core modules secure authentication with role-based access, doctor discovery (list/search), doctor profile viewing, appointment booking and history, health tracking interfaces, medical records screens, and telemedicine call functionality the system improves accessibility, organization, and overall user experience. The integration of Firebase services (Auth, Firestore, Storage) supports reliable data handling, while the inclusion of Agora RTC enables real-time telemedicine consultation, enhancing convenience for users who need remote healthcare support.

Overall, the project meets its key objectives by delivering a scalable cross-platform Flutter application with a clean multi-role architecture. It provides a strong foundation for a practical healthcare solution, while also leaving clear pathways for completing remaining integrations and expanding features toward a production-ready system.

7.2 Future Work

Although the current implementation establishes a solid base, several enhancements can further improve functionality, reliability, and real-world usability of LifeEase:

- **Complete AI-Based Healthcare Support:**
  Fully implement the AI module (currently an AiService skeleton) to provide features such as symptom guidance, basic triage suggestions, and health Q/A support with safe and controlled prompting.
- **Push Notifications (FCM) for Real Reminders:**
  Extend the current in-app notification approach to full **Firebase Cloud Messaging**

(FCM) so users receive background alerts for **appointments and medication reminders**, even when the app is closed.

- **Offline Support and Data Caching:**
  Add offline storage (e.g., Hive/SQLite) and caching so core features like viewing records, profiles, and recent appointments remain usable without internet connectivity.
- **Emergency SOS Feature:**
  Introduce an emergency module for patients, enabling quick SOS actions such as calling emergency contacts, sharing location (if allowed), or rapid access to key medical info.
- **Google Maps Full Integration:**
  Complete map-based doctor discovery using proper Google Maps integration for accurate location searching and navigation (currently marked as partial).
- **Security Hardening:**
  Improve security through features like biometric authentication, optional 2FA, stronger encryption practices, and enhanced rate-limiting/abuse protections where relevant.
- **Payments / Monetization Integration (Optional):**
  Add payment gateway support (e.g., Stripe or local payment solutions) for paid appointment booking or telemedicine services if the project scope requires monetization.
- **Testing Coverage and CI/CD:**
  Increase unit/widget/integration test coverage and add a CI pipeline (GitHub Actions) for automated builds, tests, and consistent delivery.

These enhancements will make LifeEase more complete, secure, scalable, and suitable for real-world healthcare environments, ensuring long-term effectiveness for patients, doctors, and administrators.

8 References

[1] I. Sommerville, "Software Engineering," 2015.

[2] M. Fowler, "Patterns of Enterprise Application Architecture," 2002.

[3] H. C. C. L. M. M. J. &. C. D. A. Purchase, UML Class Diagram Syntax: An Empirical Study of Comprehension. In Australasian Symposium on Information Visualisation, 2001.

[4] D. J. a. J. H. Martin, "Speech and Language Processing," 2023..

[5] G. J. Myers, "The Art of Software Testing," 2011.

[6] H. v. Vliet, Software Engineering: Principles and Practice., Wiley, 2007.

[7] K. B. e. al, "Manifesto for Agile Software Development," 2001.

[8] Auth0, "Introduction to JSON Web Tokens (JWT)," 2024.